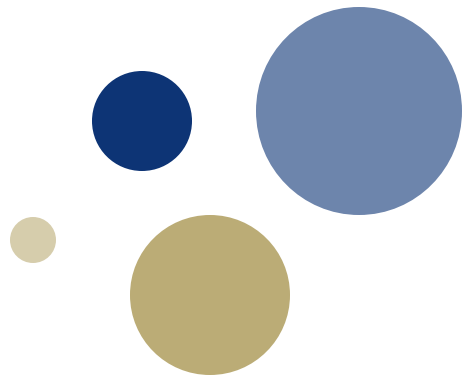




NTNU

Kunnskap for en bedre verden

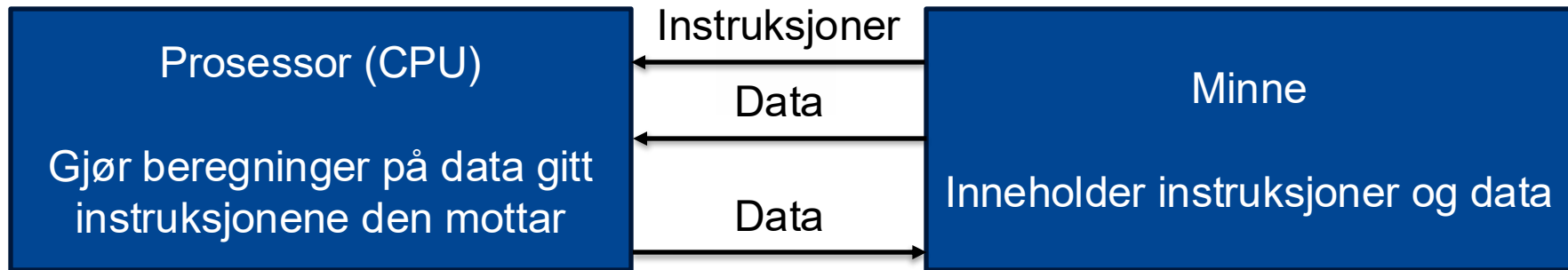


Forelesning 5: Aritmetisk-logisk enhet

TDT4160 Datamaskiner

Prinsippet om lagrede program

Dette kan vi nå

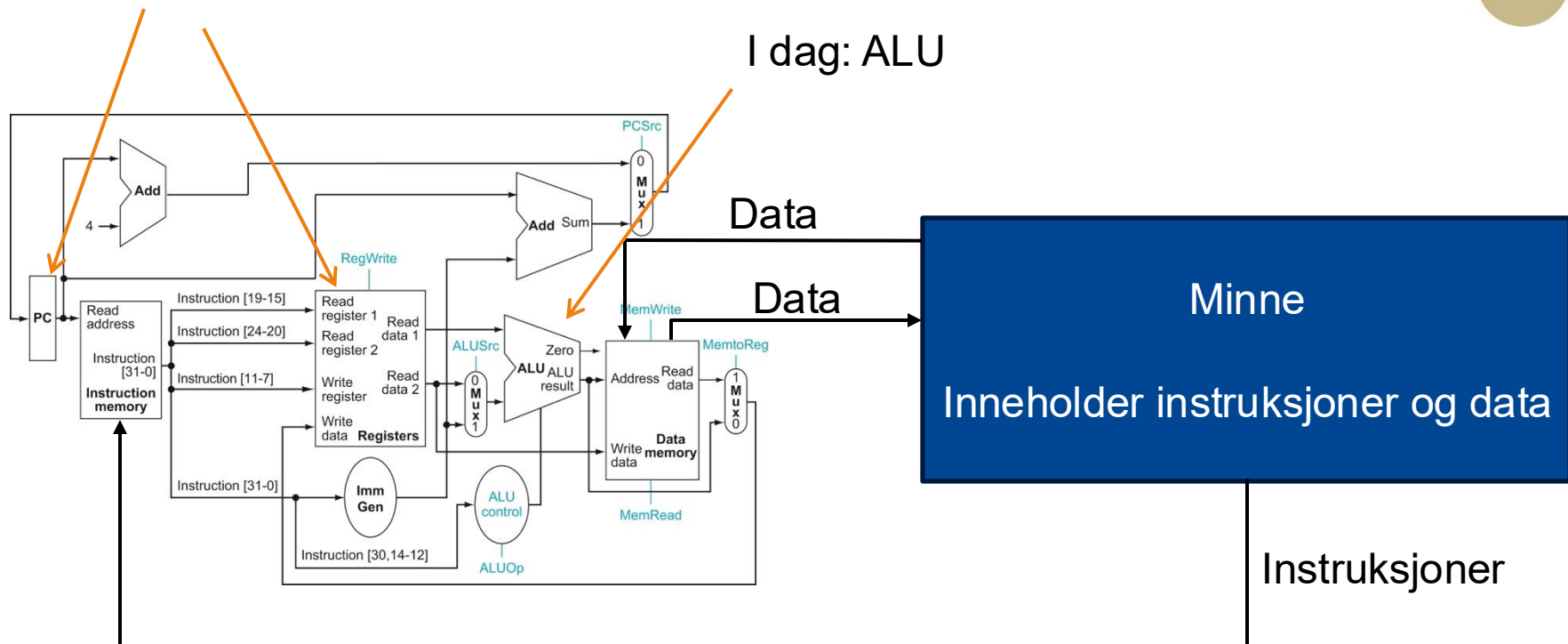


Vi vet en god del om hva som skjer inni denne

Prinsippet om lagrede program

I går: Sekvensiell logikk

I dag: ALU





Tema 3.3: Aritmetisk-logisk enhet (Kapittel 3.1, 3.2, 3.6 og A.5)

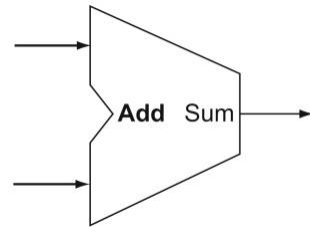
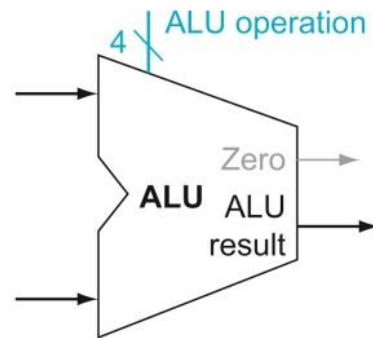
ADDISJON OG SUBTRAKSJON

Læringsutbytte T3.3

- Studenten skal kunne konstruere en 32-bit aritmetisk-logisk enhet (ALU) som kan gjøre addisjon og subtraksjon samt logisk AND og OR og nulldeteksjon (gjengi og forklare figurene A.5.7 og A.5.8).
- Studenten skal kunne utføre multiplikasjon og divisjon av binære tall og beskrive hvordan disse implementeres i maskinvare.
- Studenten skal kunne oppgi fordeler og ulemper med å representere tall i et «fixed-point» format.
- Studenten skal kunne motivere for hvorfor vi trenger flyttall og beskrive prinsippene for hvordan addisjon, subtraksjon, multiplikasjon, og divisjon med flyttallsverdier utføres i maskinvare.
- Studenten skal kunne konvertere fra desimaltall til flyttall og motsatt.
- Studenten skal kjenne til hvordan man implementerer SIMD-instruksjoner og hva de brukes til.

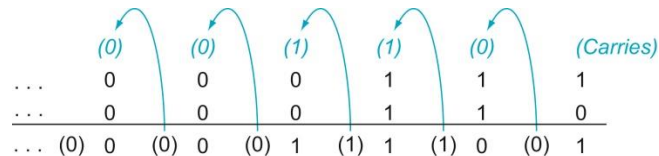
Aritmetisk-logisk enhet (ALU)

- «Arithmetic Logic Unit (ALU)» utfører ulike aritmetiske (addisjon, subtraksjon, etc.) og logiske operasjoner (and, or, etc.)
 - ... og har maskinvare for alle operasjonene den skal støtte
 - Kontrollsignalet bestemmer hvilken maskinvare som blir brukt
- Addisjonsenhetene vi har sett så langt er et eksempel på maskinvarestøtte for en aritmetisk instruksjon
 - ... og en ALU har dermed en addisjonsenhet og andre enheter inni seg



Addisjon og subtraksjon

- Vi gjør dette akkurat som for store tall i 10-tallssystemet
- $A - B$ er det samme som $A + B$ med B i 2's komplement form
 - Dette er knot nå, men gjør at vi kan legge sammen og trekke fra med nesten samme maskinvare
- Quiz!



Input		Output	
A	B	Ut	Mente
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Eksempel: Addisjon



Eksempel: Subtraksjon



Overflyt

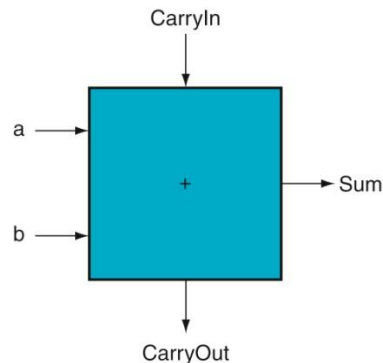
- Overflyt oppstår når vi ikke kan representere resultatet med det antallet bit vi har tilgjengelig
- Overflyt oppstår når vi:
 - Legger sammen to positive tall og får et negativt tall
 - Legger sammen to negative tall og får et positivt tall
- Når vi legger sammen et positivt tall og et negativt tall kan vi ikke få overflyt
 - ... fordi resultatet alltid er nærmere null enn operandene er

Eksempel: Overflyt



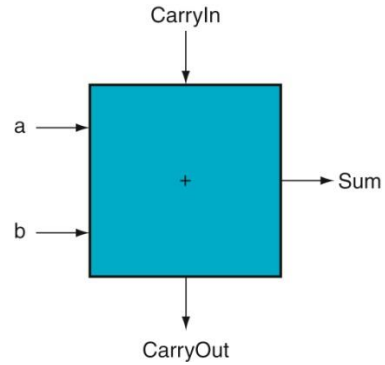
1-bit adderer

- Første skritt er å bygge maskinvare som legger sammen 1-bit verdier
- En 1-bit adderer tar inn:
 - a og b: Bitene som skal legges sammen
 - CarryIn: Mente fra forrige addisjon
- ... og produserer:
 - Sum: En-bit summen av a og b
 - CarryOut: Mente til neste addisjon



Inputs			Outputs		Comments
a	b	CarryIn	CarryOut	Sum	
0	0	0	0	0	$0 + 0 + 0 = 00_{\text{two}}$
0	0	1	0	1	$0 + 0 + 1 = 01_{\text{two}}$
0	1	0	0	1	$0 + 1 + 0 = 01_{\text{two}}$
0	1	1	1	0	$0 + 1 + 1 = 10_{\text{two}}$
1	0	0	0	1	$1 + 0 + 0 = 01_{\text{two}}$
1	0	1	1	0	$1 + 0 + 1 = 10_{\text{two}}$
1	1	0	1	0	$1 + 1 + 0 = 10_{\text{two}}$
1	1	1	1	1	$1 + 1 + 1 = 11_{\text{two}}$

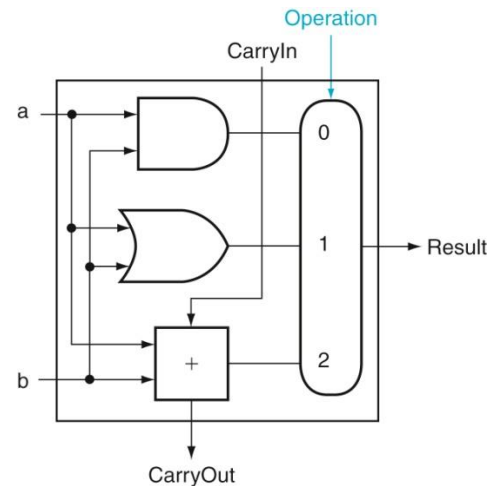
Eksempel: 1-bit adderer



Inputs			Outputs
a	b	CarryIn	Sum
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Flere operasjoner

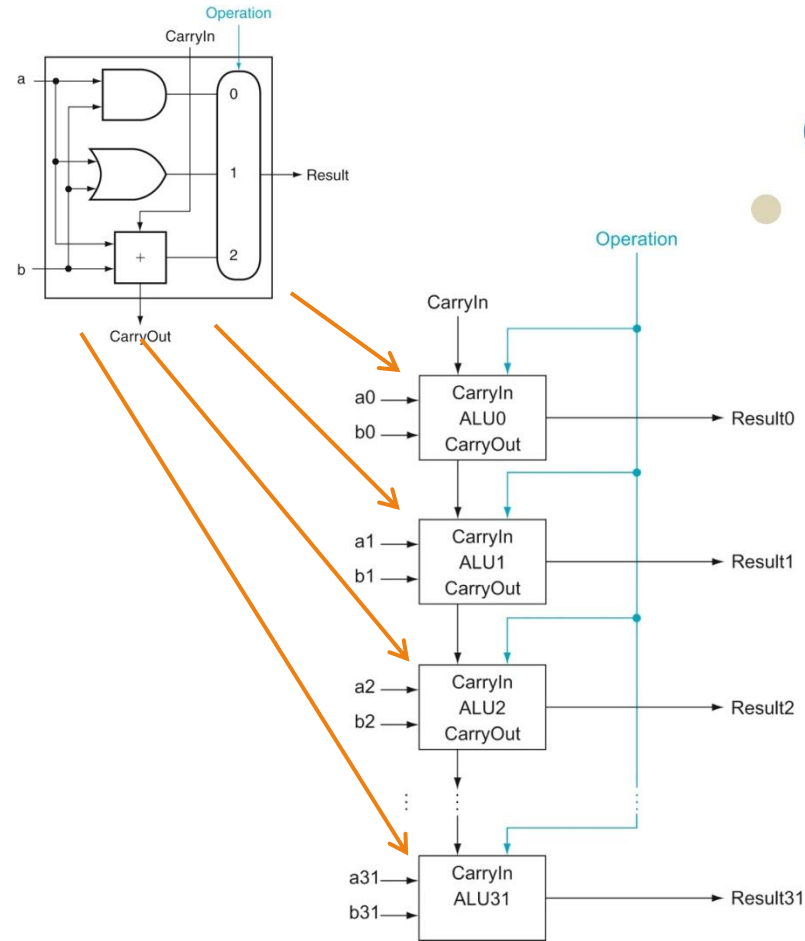
- Vi skal også støtte instruksjoner som gjør logisk `and` og logisk `or`
- Dette vet vi hvordan vi skal gjøre:
 1. Legge til maskinvare for de nye operasjonene
 2. Legge til en multiplekser for å velge resultatet fra rett enhet
 3. Eksponere inngangsverdien(e) til multiplekseren som et kontrollsignal



Figur A.5.8

32-bit ALU

- Vi kan nå lage ALUer for så mange bit som vi bare vil gjennom å koble sammen 1-bit ALUer
- CarryOut for bit n blir CarryIn på bit $n+1$
 - Akkurat som når vi regner for hånd!
 - (... men første og siste bit behandles spesielt)



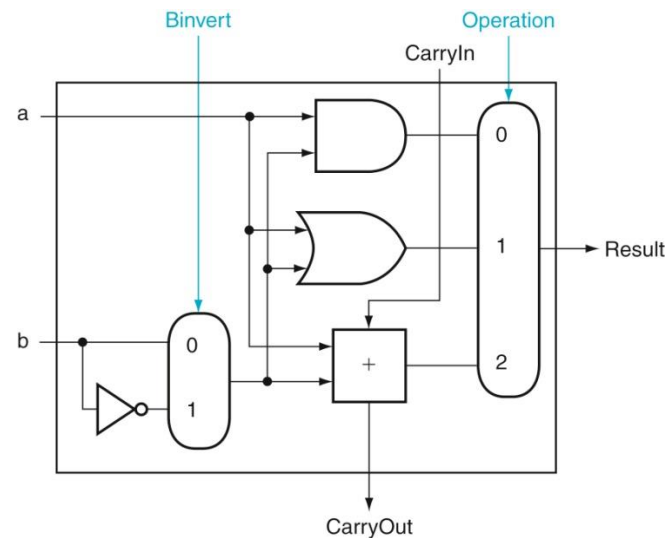
Figur A.5.7

Støtte for subtraksjon

- Nå kommer endelig 2's komplement til nytte:

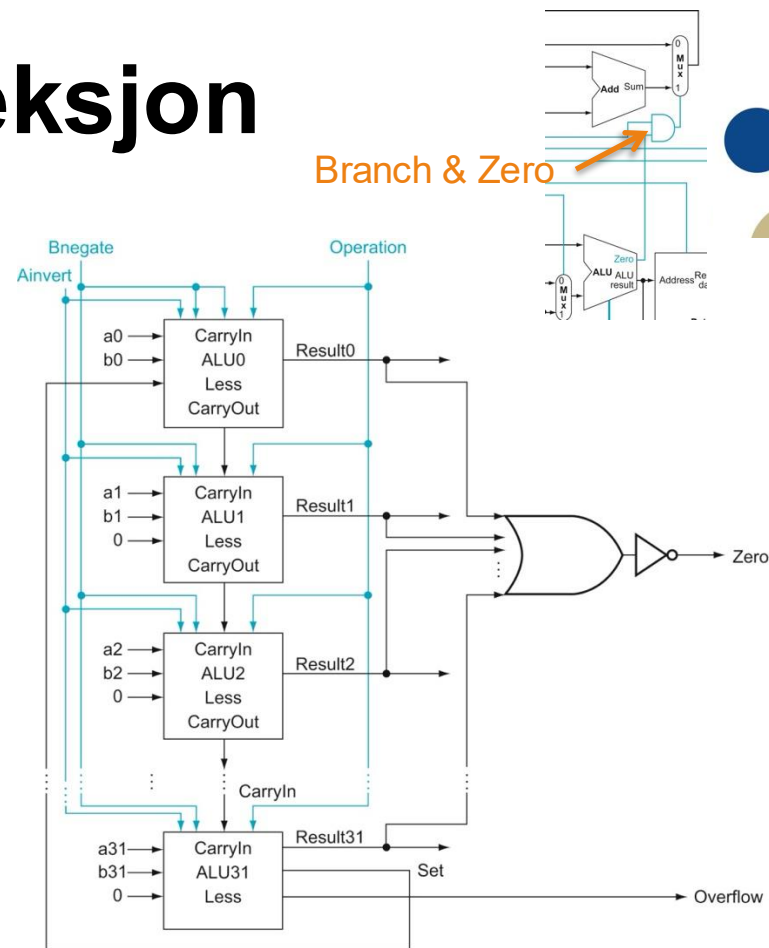
$$- A - B = A + \bar{B} + 1 \leftarrow \text{Skritt 4}$$

- Vi kan støtte subtraksjon gjennom å:
 - Legge til en inverter for B
 - Legge til en multiplexer som velger B eller $\bar{B}$
 - Ekspone inngangen til multiplexeren som et kontrollsignal
 - Sørge for at *CarryIn* på bit 0 settes til 1 ved subtraksjon

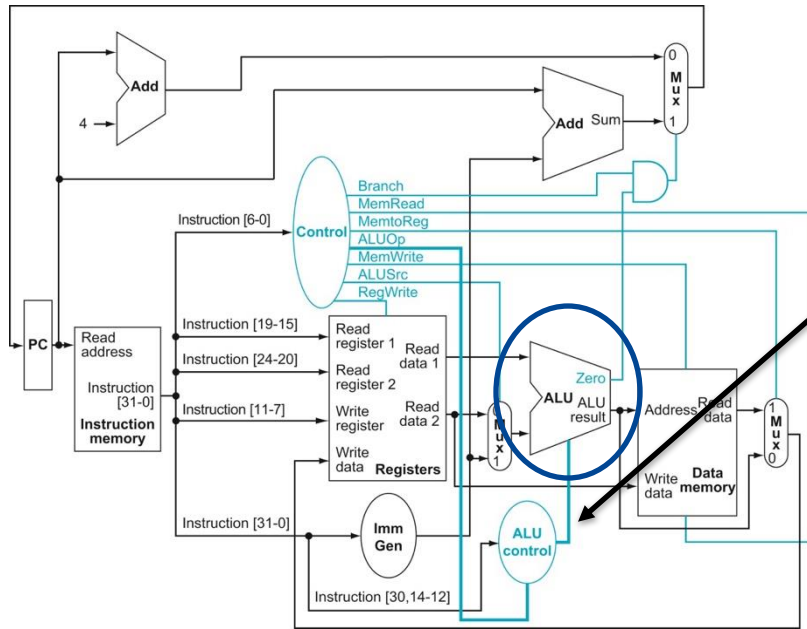


Overflyt og nulldeteksjon

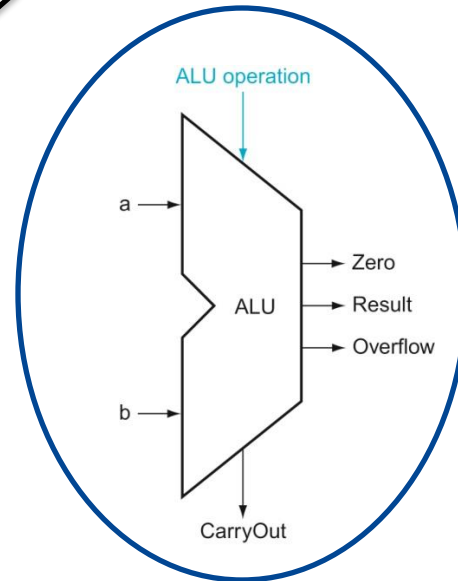
- For å støtte `beq` instruksjonen må ALUen kunne detektere at $A - B == 0$
 - Mer maskinvare: En stor OR-port og en inverter
 - Vi trenger ikke noen multiplexer fordi prosessoren bare bruker `Zero` når vi trenger den
- Vi trenger også logikk for å detektere overflyt



En fiks ferdig ALU til prosessoren

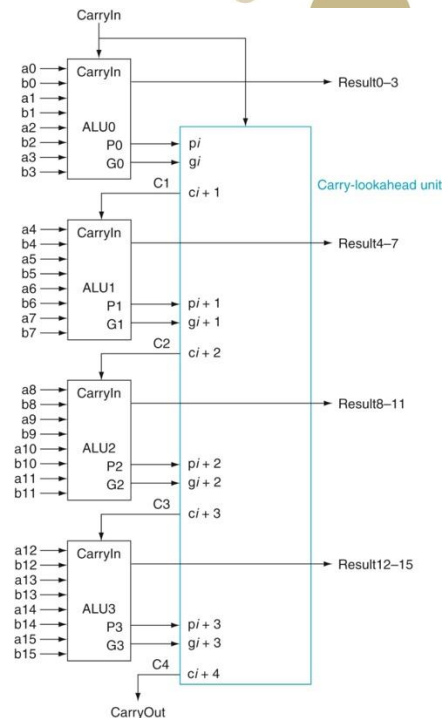
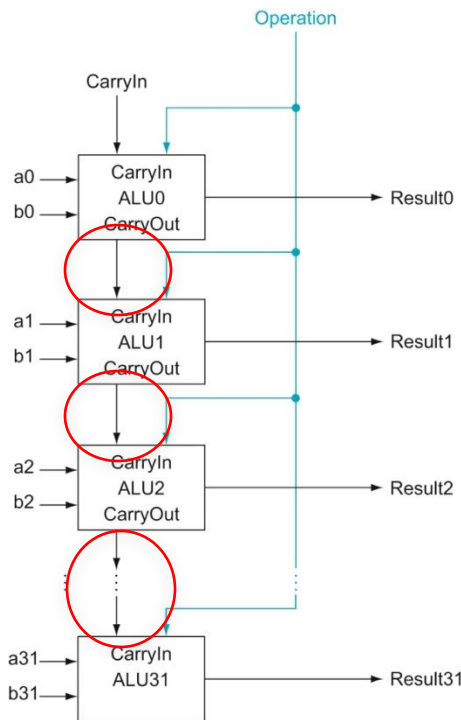


ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract



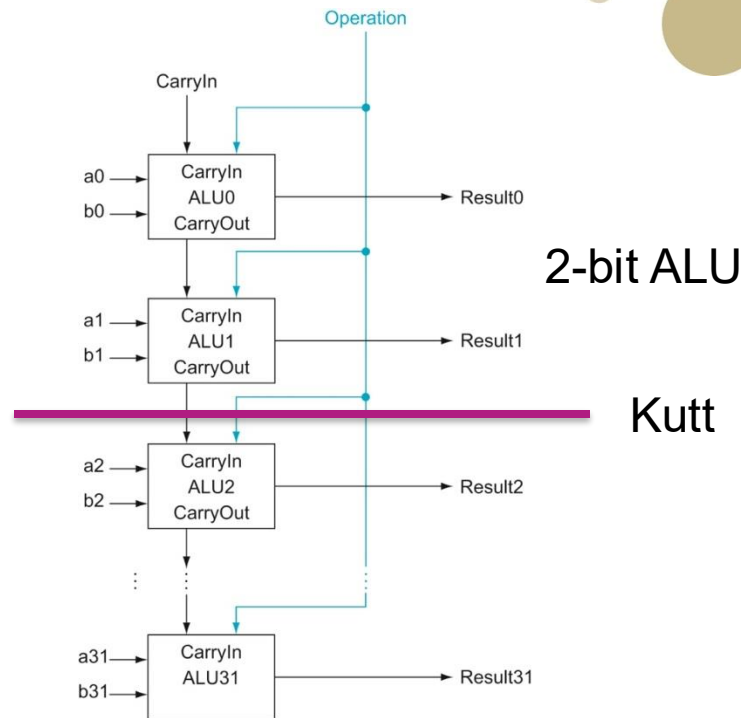
Raskere addisjon

- Addereren vi har laget kalles en «ripple-carry adder»
 - ... og her må vi sette av nok tid til at CarryIn til bit 0 propagerer gjennom alle 1-bit addererne
 - Vi regner altså ut menteverdiene *sekvensielt* og dette kan være et problem fordi lange signalveier kan begrense klokkefrekvensen
- Alternativ: «Carry look-ahead adder»
 - Legger til mer maskinvare for å regne ut menteverdiene samtidig (*parallelt*)
- Addisjon tar typisk én klokkesykel i høytytelsesprosessorer



Triks: SIMD-instruksjoner

- Mange bruksområder (for eksempel lyd og bilde) trenger ikke 32 bit til å representere verdiene sine
 - ... og har derfor bruk for smalere ALUer
- Vi kan lage smalere ALUer av en bred ALU ved å legge til logikk som deler den opp
 - ... og lage instruksjoner som gjør flere operasjoner i parallell
 - (Mye) bedre ytelse for en liten ekstrakostnad
 - Slike instruksjoner kalles SIMD-instruksjoner (Single-Instruction Multiple Data)





T3.3: Aritmetisk-logisk enhet (Kapittel 3.3 og 3.4)

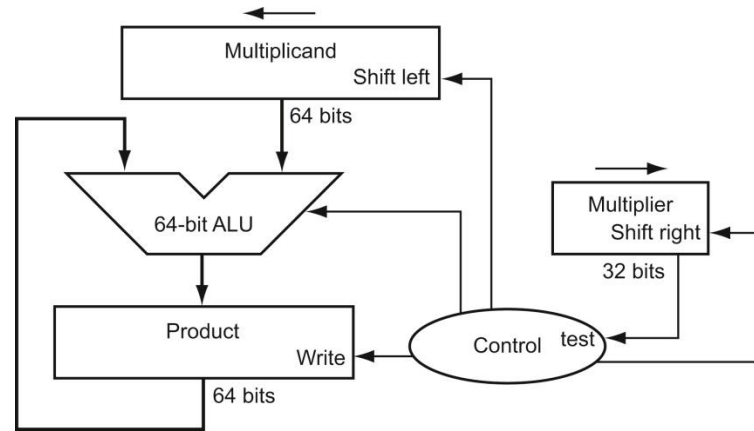
MULTIPLIKASJON OG DIVISJON

Læringsutbytte T3.3

- Studenten skal kunne konstruere en 32-bit aritmetisk-logisk enhet (ALU) som kan gjøre addisjon og subtraksjon samt logisk AND og OR og nulldeteksjon (gjengi og forklare figurene A.5.7 og A.5.8).
- **Studenten skal kunne utføre multiplikasjon og divisjon av binære tall og beskrive hvordan disse implementeres i maskinvare.**
- Studenten skal kunne oppgi fordeler og ulemper med å representere tall i et «fixed-point» format.
- Studenten skal kunne motivere for hvorfor vi trenger flyttall og beskrive prinsippene for hvordan addisjon, subtraksjon, multiplikasjon, og divisjon med flyttallsverdier utføres i maskinvare.
- Studenten skal kunne konvertere fra desimaltall til flyttall og motsatt.
- Studenten skal kjenne til hvordan man implementerer SIMD-instruksjoner og hva de brukes til.

Multiplikasjon

- Vi kan se på multiplikasjon som en serie av addisjonsoperasjoner
- Vi kan bygge maskinvare som gjør dette
 - ... og hvis vi ikke har maskinvare-støtte for multiplikasjon kan vi (kompilatoren) generere en løkke med `add` instruksjoner
 - Hvis vi skal gange med en toerpotens, kan vi bruke skiftinstruksjoner

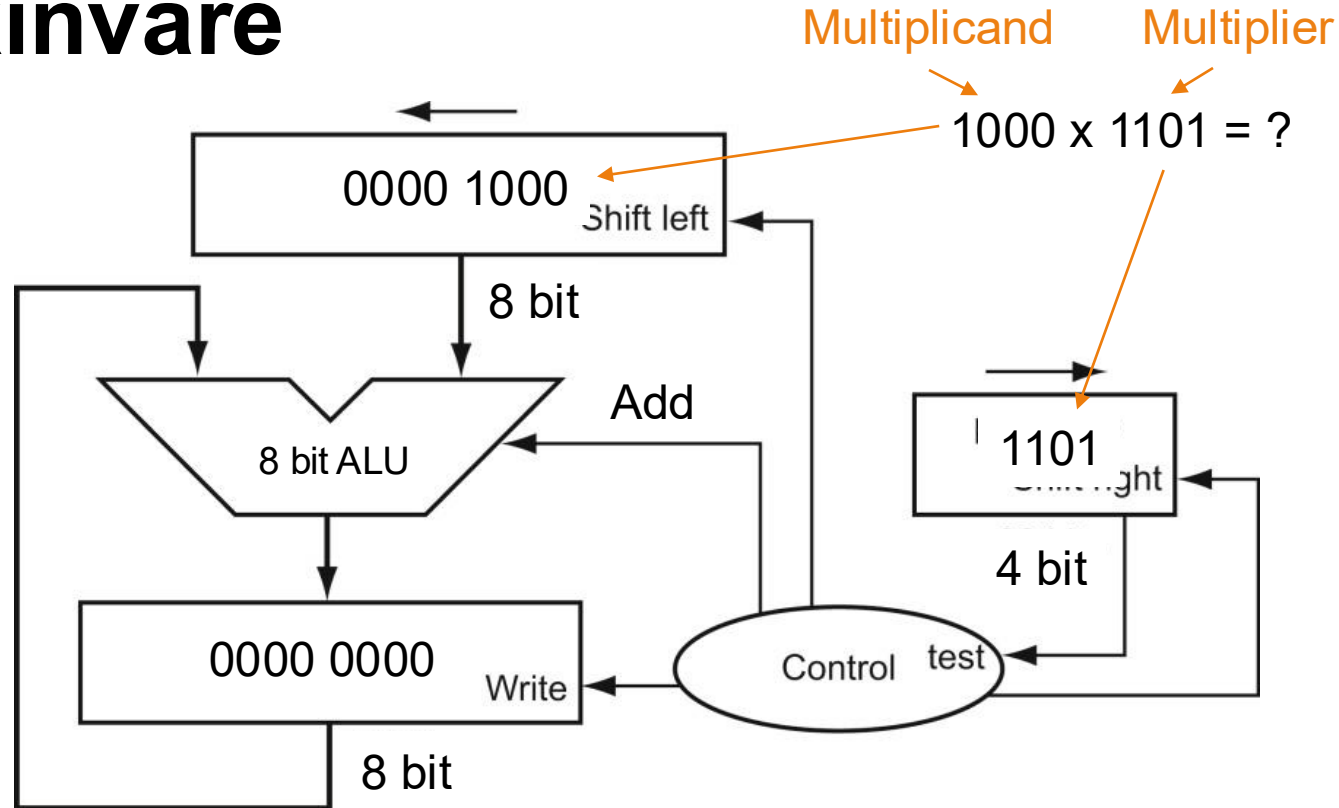


Eksempel: Multiplikasjon for hånd



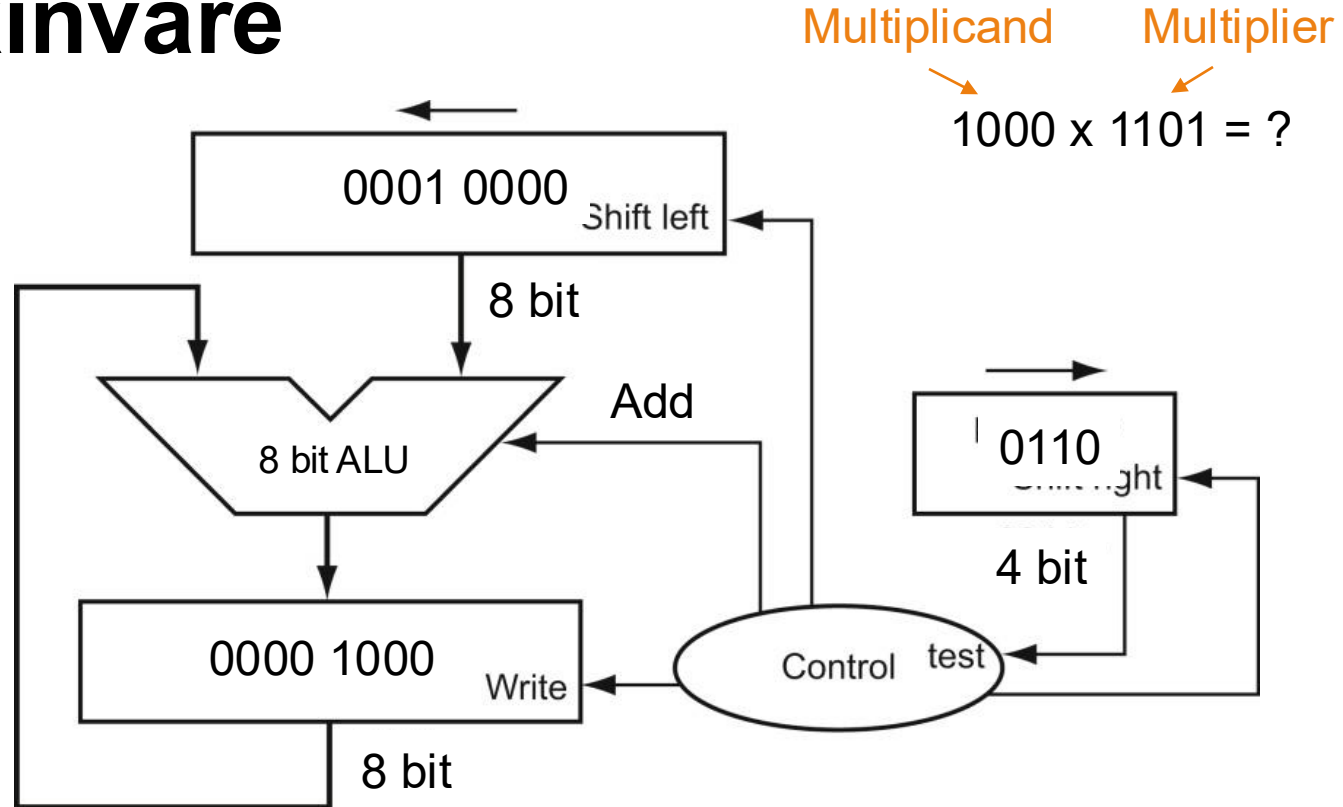
Eksempel: Multiplikasjon i maskinvare

Start skritt 0



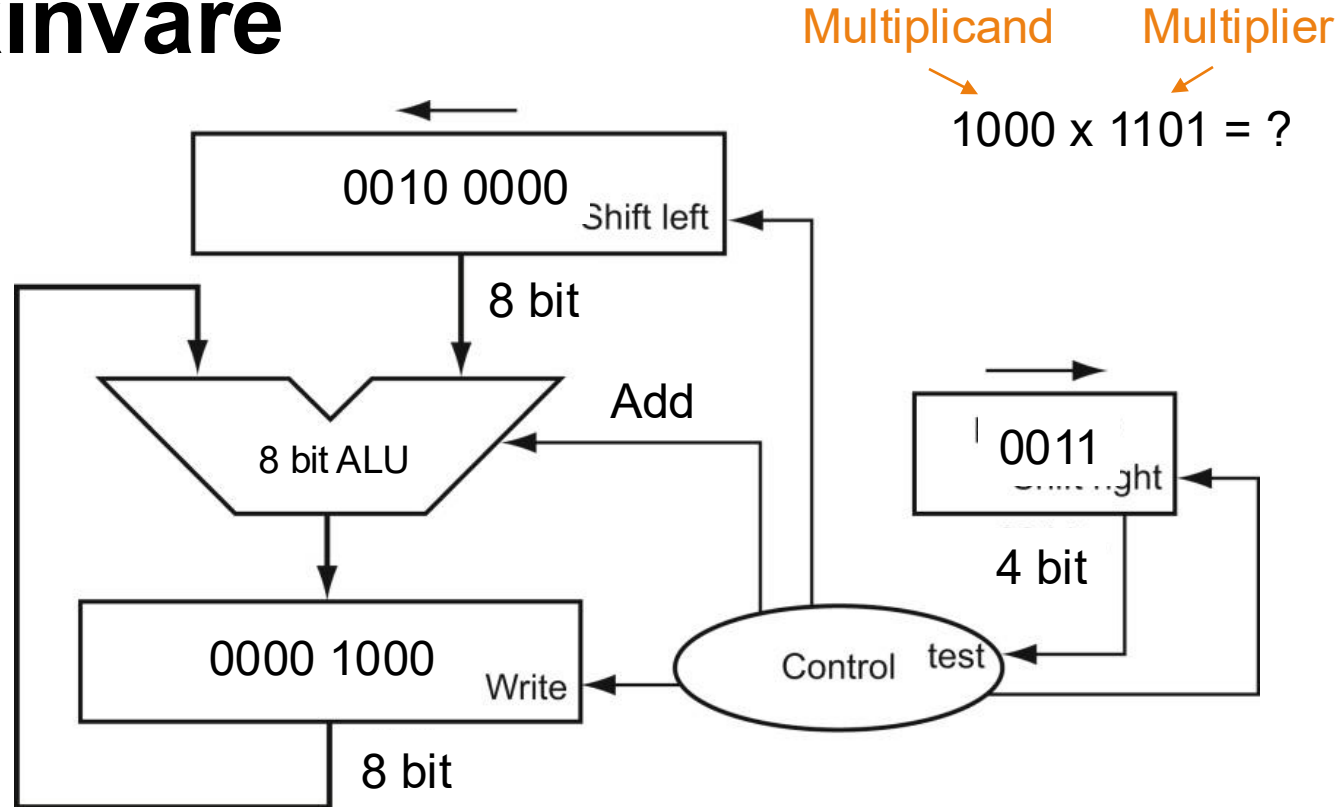
Eksempel: Multiplikasjon i maskinvare

Start skritt 1



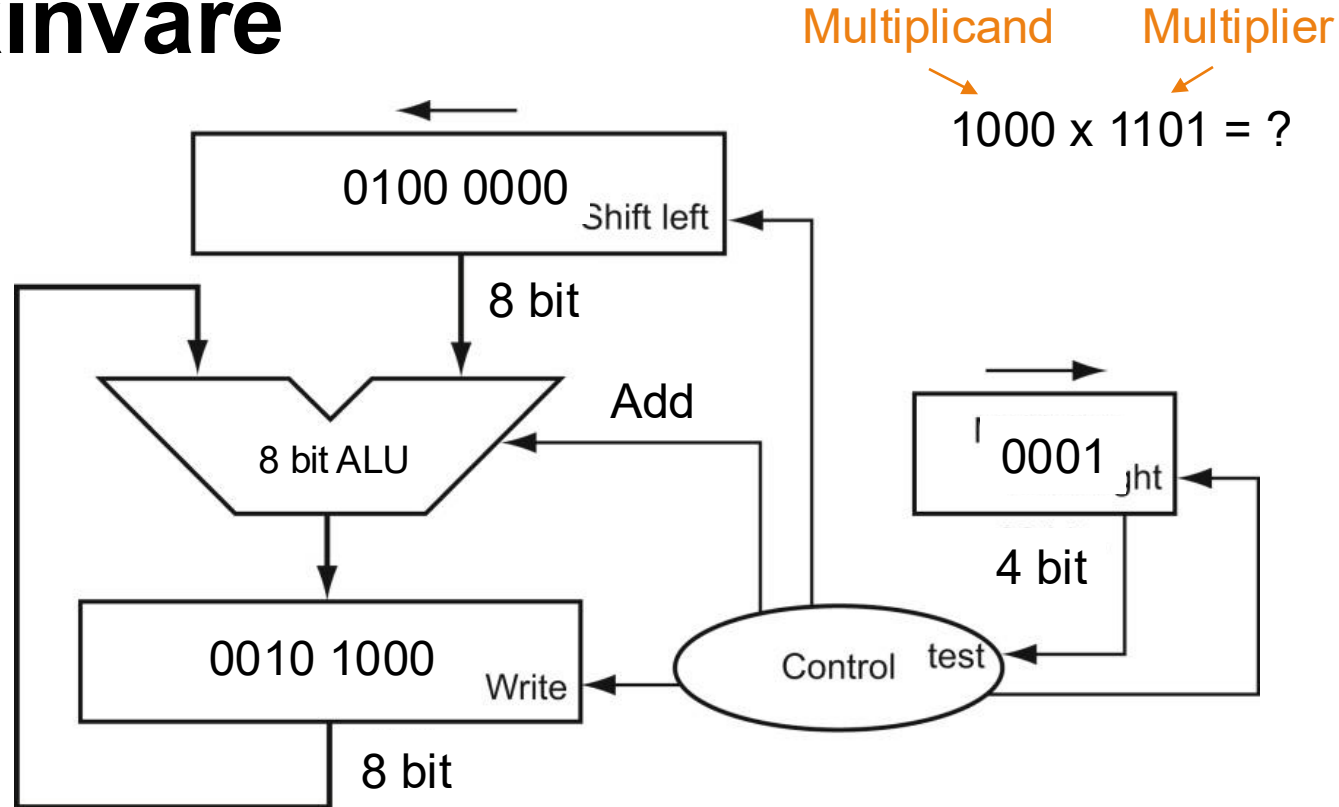
Eksempel: Multiplikasjon i maskinvare

Start skritt 2



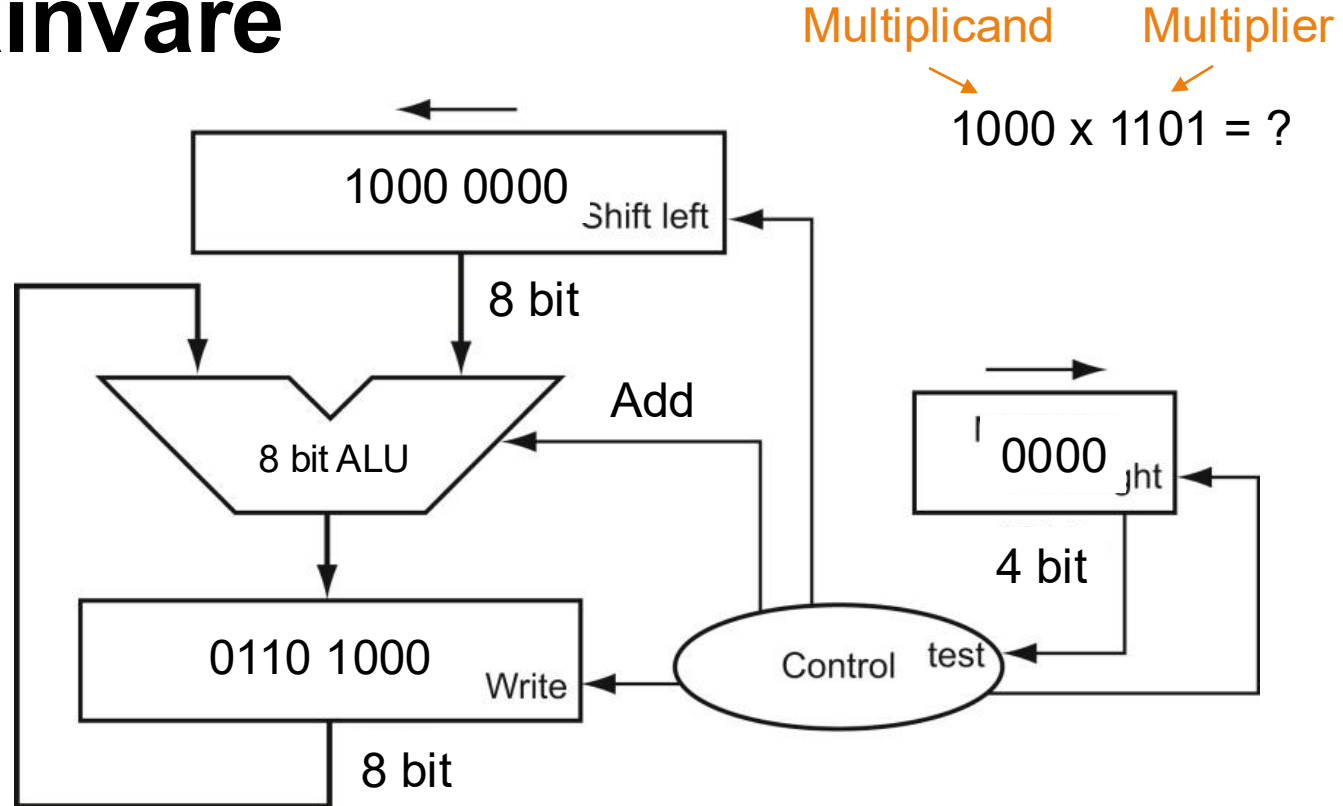
Eksempel: Multiplikasjon i maskinvare

Start skritt 3



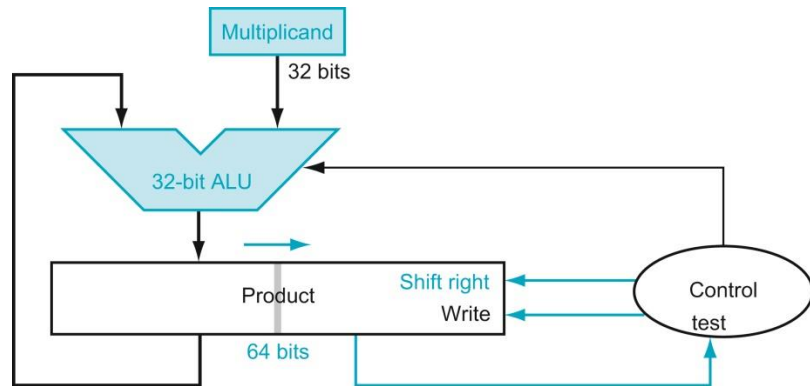
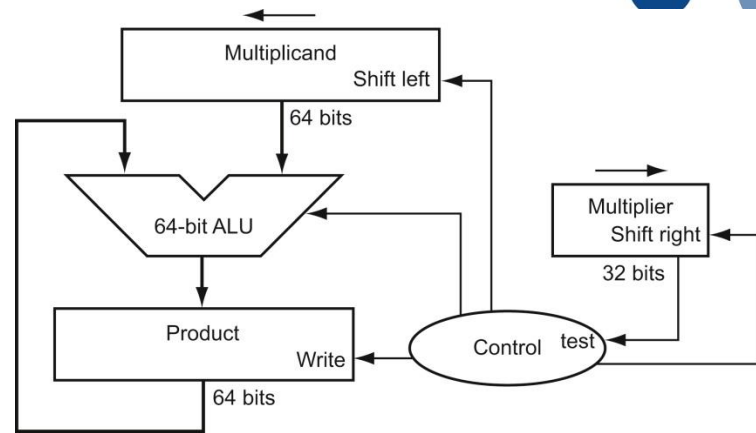
Eksempel: Multiplikasjon i maskinvare

Slutt skritt 3



Litt raskere multiplikasjon

- Forrige versjon bruker flere klokkesykler på hvert skritt
 - ... og mange av bitene i ALUen og registrene er ikke i bruk
- Løsning:
 - Gjøre skifting og addisjon i parallell
 - Halvere størrelsen på ALU og «Multiplicand»
 - Legge «Multiplier» i nedre halvdel av «Product»



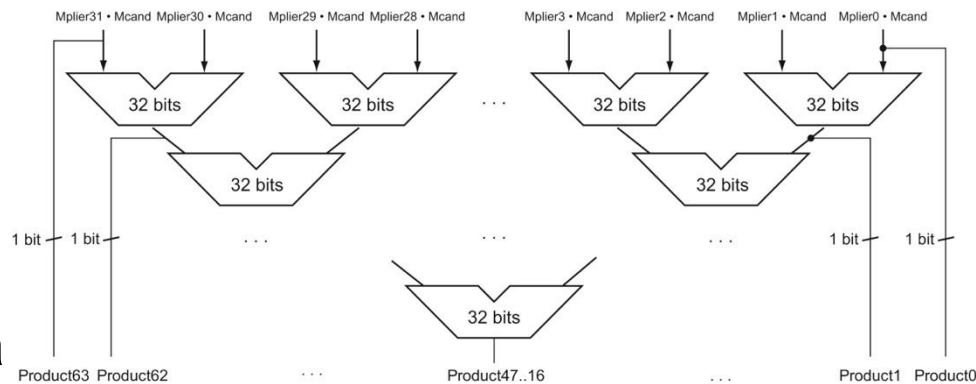
Enda raskere multiplikasjon

- For å få multiplikasjon til å gå enda raskere må vi gjøre mer i parallell

- Løsning: Et tre av adderere
- Multiplikasjon tar typisk ~3 klokkesyker i høytelesprosessorer

- Vi håndterer fortegn/bit huske fortegnene og korrigere etterpå

- ++ og -- gir +
- +- og -+ gir -



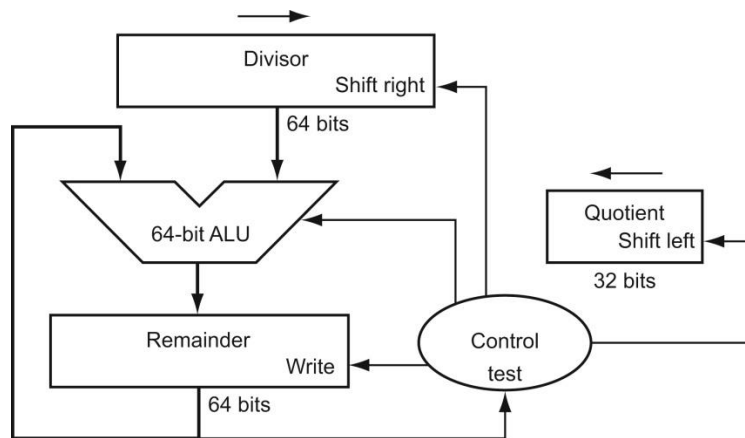
Divisjon

- Vi implementerer divisjon etter de samme prinsippene som multiplikasjon
 - En løkke med subtraksjon
 - ... men vi skal bare trekke fra så lenge restverdien er større enn 0
- Vi må også håndtere at divisjon med 0 ikke er lov (avbrudd)

Input $\xrightarrow{\text{Dividend} - \text{Rest}}$ $\xrightarrow{\text{Divisor}}$ Output Kvotient

$$\frac{\text{Dividend} - \text{Rest}}{\text{Divisor}} = \text{Kvotient}$$

$$\text{Dividend} = \text{Kvotient} \cdot \text{Divisor} + \text{Rest}$$

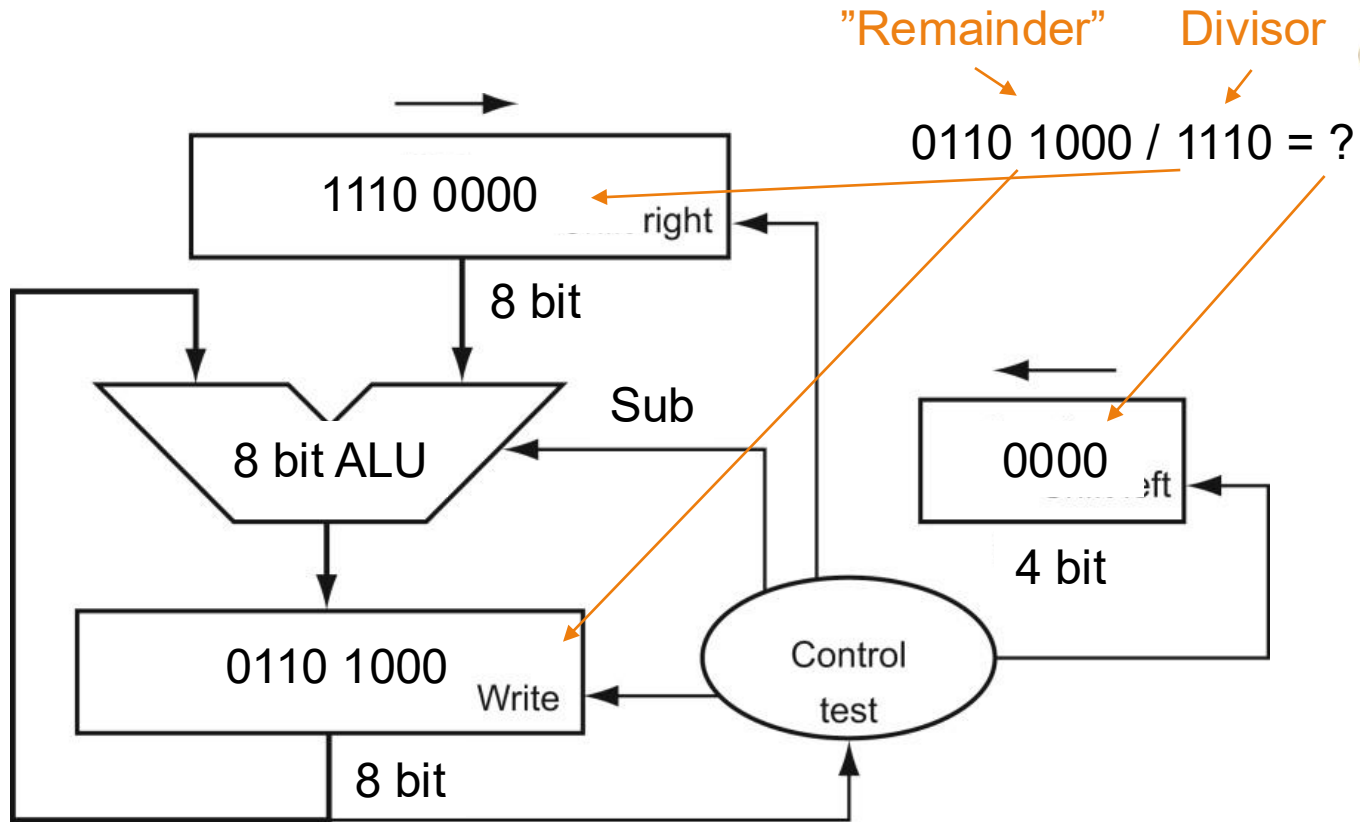


Eksempel: Divisjon for hånd



Eksempel: Divisjon i maskinvare

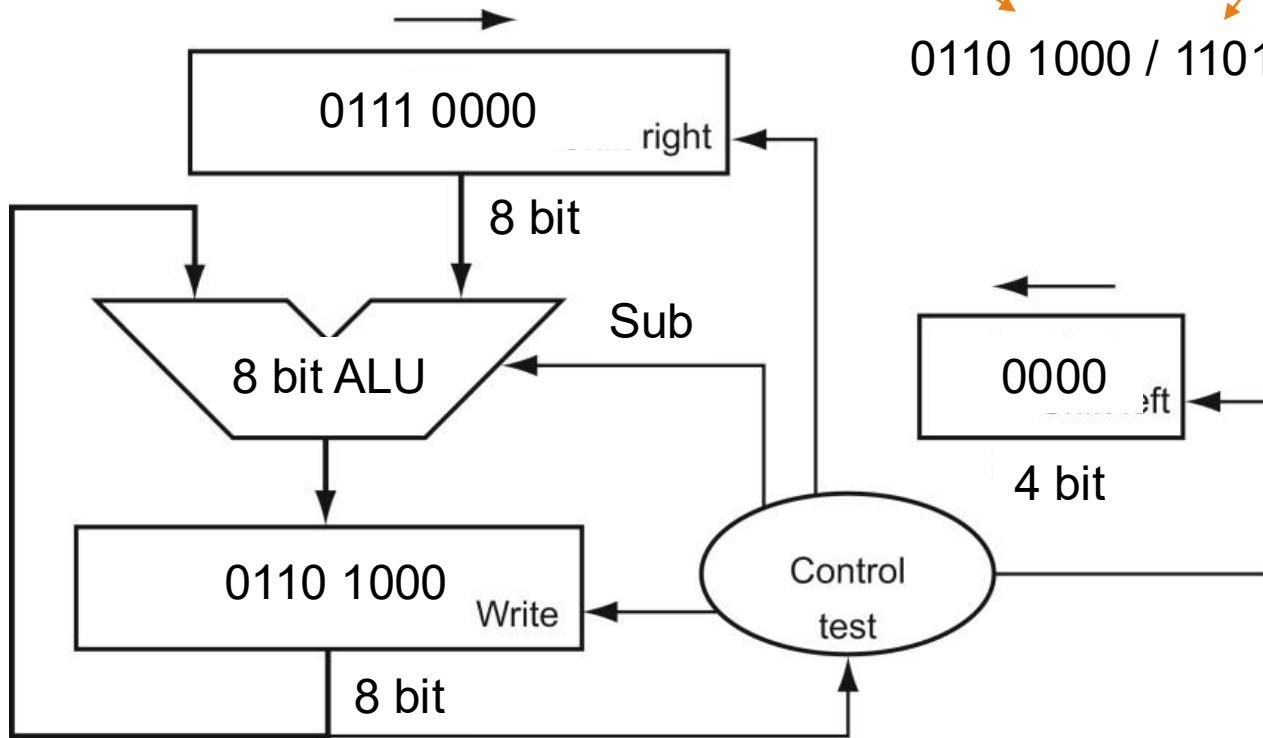
Start skritt 0



Eksempel: Divisjon i maskinvare

"Remainder" Divisor
0110 1000 / 1101 = ?

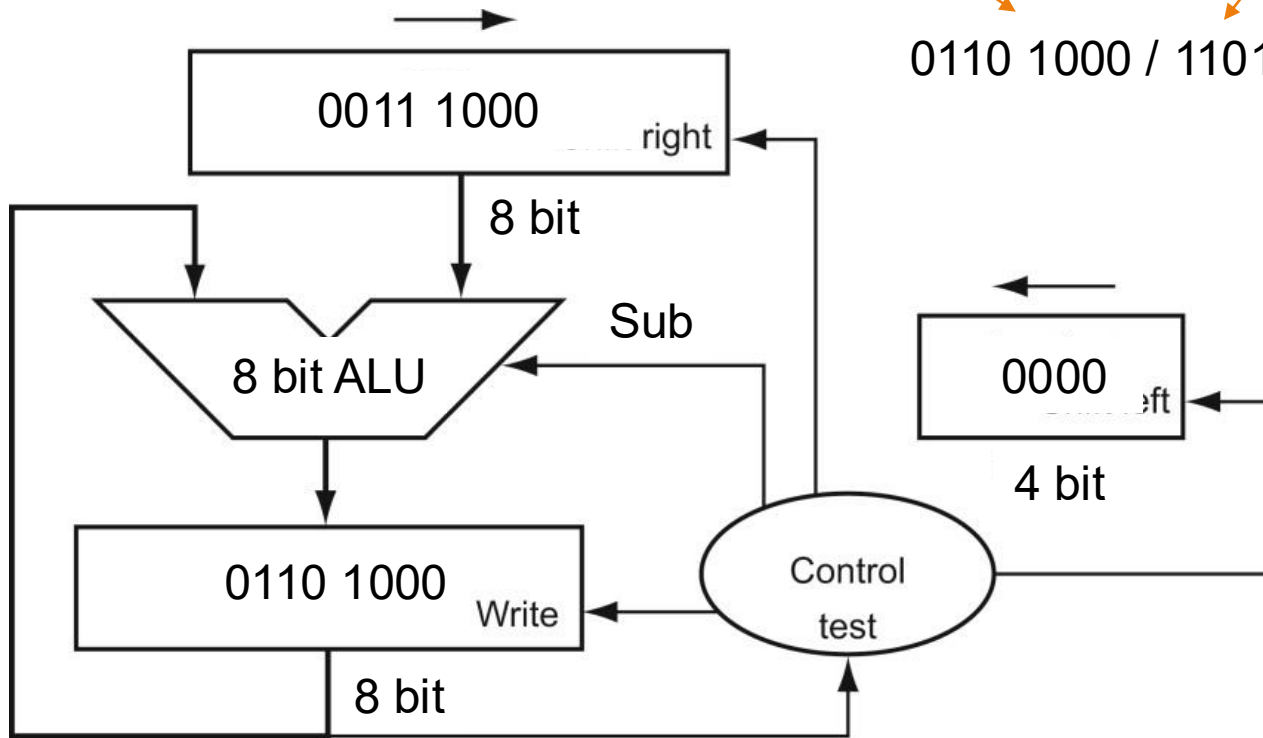
Start skritt 1



Eksempel: Divisjon i maskinvare

"Remainder" Divisor
0110 1000 / 1101 = ?

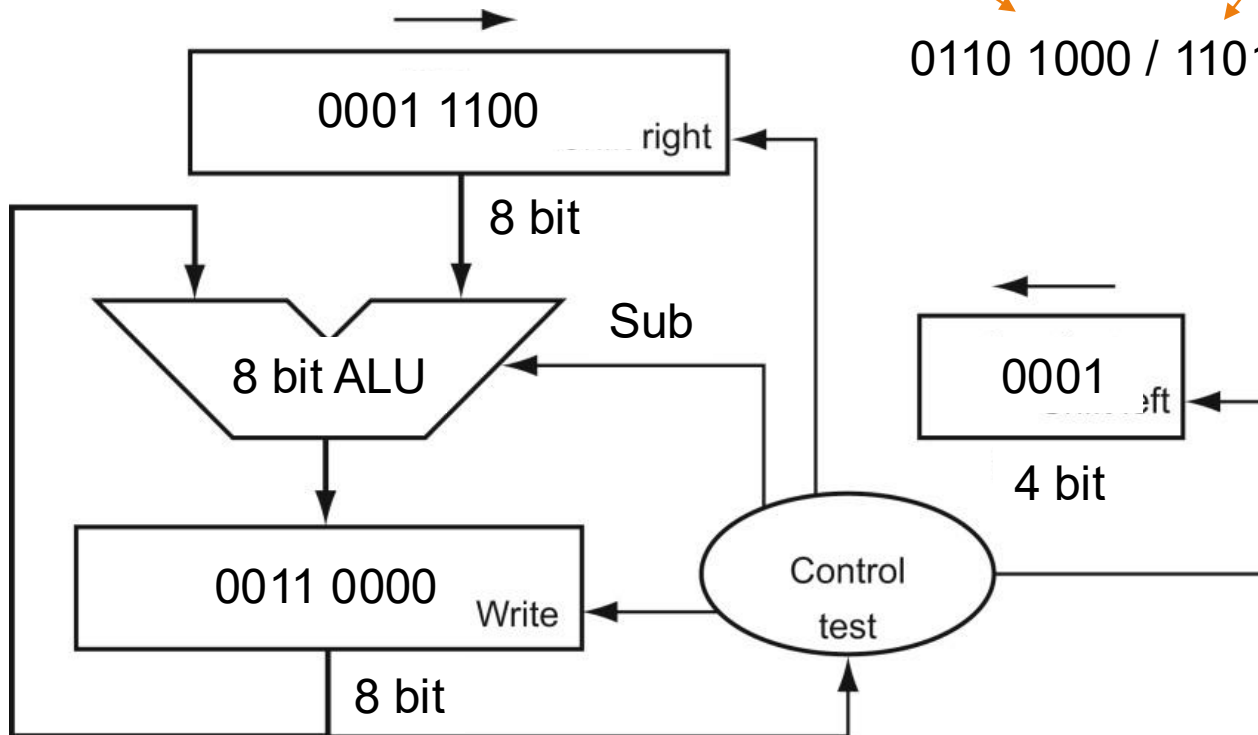
Start skritt 2



Eksempel: Divisjon i maskinvare

"Remainder" Divisor
0110 1000 / 1101 = ?

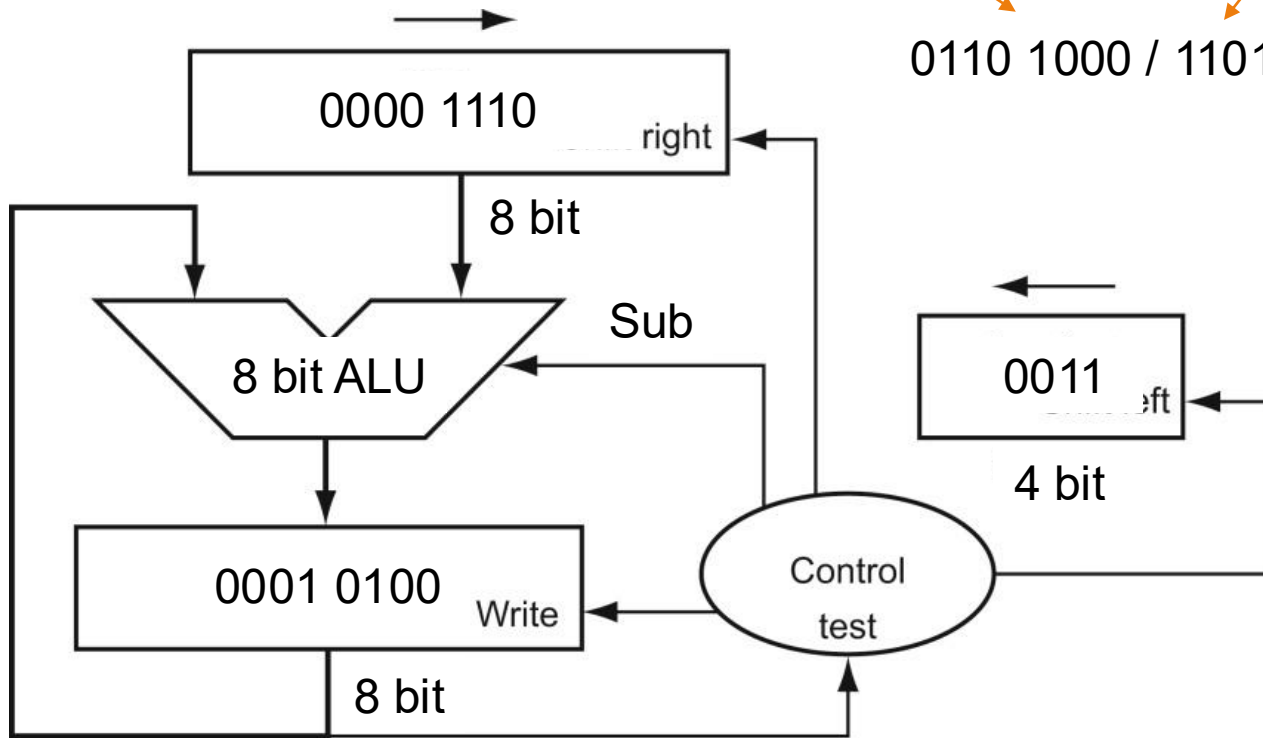
Start skritt 3



Eksempel: Divisjon i maskinvare

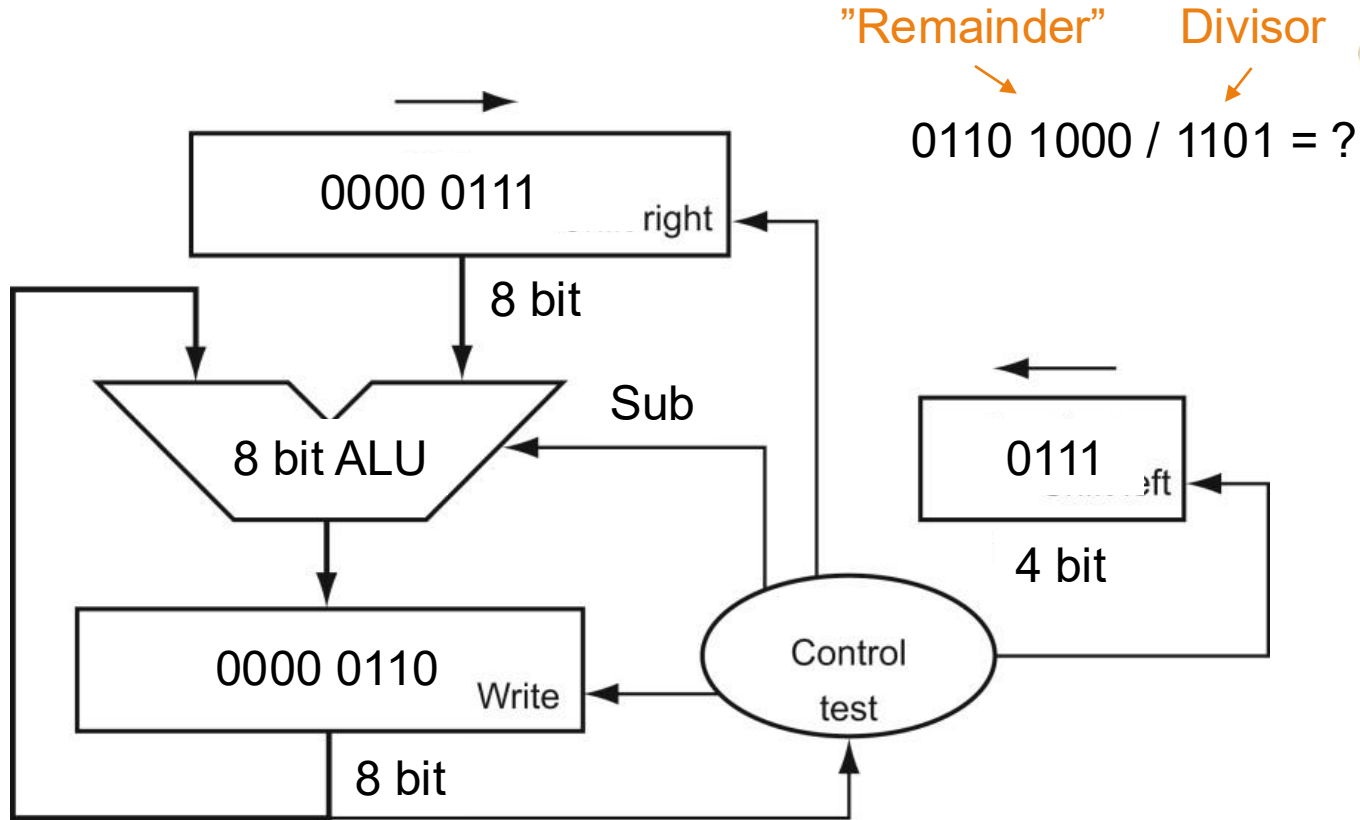
"Remainder" Divisor
0110 1000 / 1101 = ?

Start skritt 4



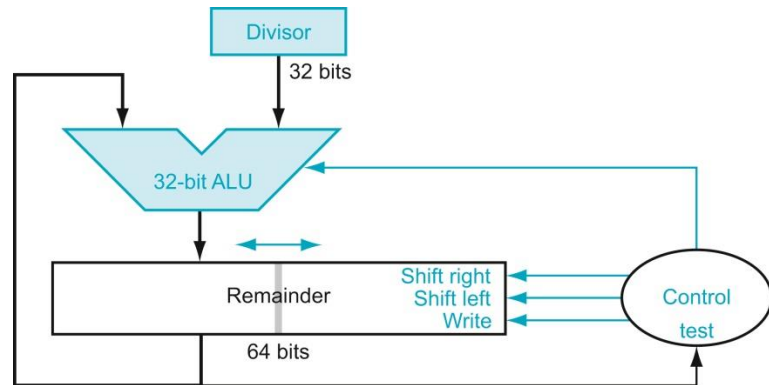
Eksempel: Divisjon i maskinvare

Slutt skritt 4



Raskere divisjon

- Vi kan gjøre tilsvarende optimalisering som for multiplikasjon for å få et skritt per klokkesykel
- Problem: Hvert skritt trenger den nye verdien i «Remainder» registeret
 - Denne *avhengigheten* begrenser hvor mye vi kan parallellisere
 - Divisjon tar derfor noen titalls (~20-~40) klokkesykler på høytelesprosessorer





T3.3: Aritmetisk-logisk enhet (Kapittel 3.5)

FLYTTALL

Læringsutbytte T3.3

- Studenten skal kunne konstruere en 32-bit aritmetisk-logisk enhet (ALU) som kan gjøre addisjon og subtraksjon samt logisk AND og OR og nulldeteksjon (gjengi og forklare figurene A.5.7 og A.5.8).
- Studenten skal kunne utføre multiplikasjon og divisjon av binære tall og beskrive hvordan disse implementeres i maskinvare.
- Studenten skal kunne oppgi fordeler og ulemper med å representere tall i et «fixed-point» format.
- Studenten skal kunne motivere for hvorfor vi trenger flyttall og beskrive prinsippene for hvordan addisjon, subtraksjon, multiplikasjon, og divisjon med flyttallsverdier utføres i maskinvare.
- Studenten skal kunne konvertere fra desimaltall til flyttall og motsatt.
- Studenten skal kjenne til hvordan man implementerer SIMD-instruksjoner og hva de brukes til.

Hvorfor flyttall?

- Så langt har vi bare dekket heltall, men vi må også kunne representere desimaltall
- Det enkleste er «Fixed point»
 - Fordel: Vi kan regne som med heltall. Eneste endring er at vi omdefinierer verdien til hver bit.
 - Ulempe: Vi må velge mellom å kunne representere store tall og små tall
- Løsning: Flyttall («floating point numbers»)
 - ... altså tallformat der komma flytter seg ved behov (flyter)

Eksempel: «Fixed-point»



Flyttall

- Basert på vitenskapelig notasjon
 - $1123 = 1,123 \cdot 10^3$
 - $0,001123 = 1,123 \cdot 10^{-3}$
 - → Gjennom å endre på eksponenten kan vi representere både (veldig) store og (veldig) små tall

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s	exponent								fraction																						
1 bit	8 bits								23 bits																						

$$(-1^s) \cdot \text{fraction} \cdot 2^{\text{exponent}}$$

Presisjon og rekkevidde

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s	exponent								fraction																						
1 bit	8 bits								23 bits																						

$$(-1)^s \cdot \text{fraction} \cdot 2^{\text{exponent}}$$

- Hvor mange bit skal vi bruke på hvert felt?
 - Flere «fraction» bit øker presisjon → Vi kan representere tall mer nøyaktig
 - Flere «exponent» bit øker «range» → Vi kan representere større og mindre tall
- Flyttall er også en endelig representasjon
 - Overflyt («overflow») oppstår når tallet er for stort til å kunne representeres med «exponent»
 - Underflyt («underflow») oppstår når tallet er for lite til å kunne representeres med «exponent»

Flyttallstandarden IEEE 754

- De mest vanlige formatene i IEEE 754 er:
 - 32-bit «single precision»
 - 64-bit «double precision»
- Bias er 127 (1023) for enkel (dobbel) presisjon
 - Forenkler sortering fordi større eksponent alltid betyr større tall
- Mye kan gå galt
 - For eksempel er ikke alltid $(A + B) + C = A + (B + C)$

Eksempel!

Single precision format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s		exponent								fraction																					
1 bit		8 bits								23 bits																					

$$(-1)^s \cdot (1 + \text{Fraction}) \cdot 2^{(\text{Exponent} - \text{Bias})}$$

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	$\pm$ denormalized number
1–254	Anything	1–2046	Anything	$\pm$ floating-point number
255	0	2047	0	$\pm$ infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

Eksempel: Flyttall

Single precision format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s	exponent								fraction																						
1 bit	8 bits								23 bits																						

$$(-1)^s \cdot (1 + \text{Fraction}) \cdot 2^{(\text{Exponent} - 127)}$$

Konverter til desimal:

Fortegn	«Exponent»	«Fraction»
1	1000 0000	1000 0000 ...

Eksempel: Flyttall

Single precision format

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
s	exponent								fraction																						
1 bit	8 bits								23 bits																						

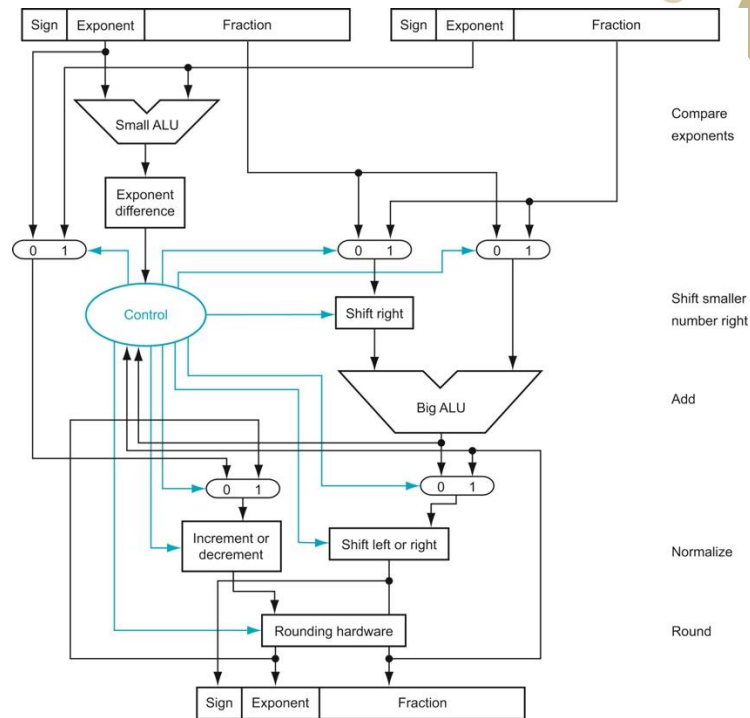
$$(-1)^s \cdot (1 + \text{Fraction}) \cdot 2^{(\text{Exponent} - 127)}$$

Representer 18,5 som et single-precision flyttall.



Maskinvare for flyttallsaddisjon

- Maskinvare for flyttallsaddisjon er komplisert men essensen er:
 - Regn ut forskjellen på eksponentene
 - Skift den miste operanden mot høyre for å få komma på samme sted
 - Legg sammen operandene
 - Skift resultatet slik at «Fraction» kommer på formen 1.xxx (normalisering)
- I høytytelsesprosessorer:
 - Flyttallsaddisjon og flyttallsmultiplikasjon tar ~4 klokkesykler
 - Flyttallsdivisjon tar fortsatt noen titalls (>20) klokkesykler





OPPSUMMERING

Hva kan vi nå?

Neste skritt: Mer effektive prosessorer

Nå vet vi hvordan disse virker

